

МИНУТА 1 — приветствие, тема, одно предложение про актуальность

[Слайд 1] → переход на [Слайд 2]

Здравствуйте, уважаемые члены комиссии! Меня зовут Гневнов Артём, я студент 4 курса направления 09.03.01 «Информатика и вычислительная техника». Тема моей выпускной квалификационной работы — «**Разработка AI-сервиса техподдержки**». Научный руководитель — кандидат физико-математических наук, доцент кафедры ИТиЭО Жуков Николай Николаевич.

[Слайд 2 — Актуальность]

Актуальность работы обусловлена потребностью организаций в AI-сервисе технической поддержки, который функционирует на собственной инфраструктуре без передачи пользовательских данных внешним провайдерам и при этом легко встраивается в существующую систему управления заявками. Готовые облачные решения такому критерию не удовлетворяют, а open-source платформы требуют значительных затрат на настройку. В качестве системы-хоста для апробации выбран программный комплекс Helperbot, разрабатываемый для ЦБС Петроградского района Санкт-Петербурга.

МИНУТЫ 2–3 — цель и задачи, коротко, без пересказа

[Слайд 3] → [Слайд 4] → [Слайд 5]

[Слайд 3 — Объект и предмет]

Объектом исследования является процесс автоматизированной обработки обращений пользователей в системе технической поддержки. **Предмет** — программные методы разработки AI-сервиса в микросервисной архитектуре с применением локальных языковых моделей.

[Слайд 4 — Цель]

Цель работы — разработать микросервис AI-технической поддержки, обеспечивающий автоматическую обработку обращений на основе локальной языковой модели, с сохранением истории диалога, ограничением нагрузки и интеграцией в систему управления заявками.

[Слайд 5 — Задачи]

Для достижения цели поставлены шесть задач: **первая** — сравнительный анализ существующих AI-решений; **вторая** — формулировка функциональных и нефункциональных требований; **третья** — проектирование архитектуры микросервиса helperbot-ai и протокола его интеграции; **четвёртая** — реализация самого микросервиса: gRPC-интерфейс, хранение диалогов в PostgreSQL, вызов локальной языковой модели через Ollama; **пятая** — интеграция AI-сервиса с сервисом бизнес-логики helperbot-core и Telegram-ботом helperbot-tg; **шестая** — тестирование и описание процедуры развёртывания через Docker Compose.

МИНУТЫ 4–6 — что сделано и что получилось (главная часть)

[Слайд 6] → [Слайд 7]

Перейду к **технологическим решениям**. В качестве языка реализации выбран **Go**: его горутины эффективно обслуживают конкурентные gRPC-вызовы, а статическая компиляция в один бинарник упрощает контейнеризацию. Межсервисное взаимодействие построено на **gRPC с Protocol Buffers** и декларативной валидацией через `protoc-gen-validate` — это исключает рассогласование контрактов между клиентом и сервером. Хранилище — **PostgreSQL** в схеме мастер-реплика, доступ через драйвер `pgx`. Сборка приложения и внедрение зависимостей — фреймворк **Uber FX**. Языковая модель запускается локально через **Ollama**; в качестве базовой выбрана `qwen2.5:1.5b` — компактная модель, дающая приемлемое качество на русском языке в пределах ограничения контейнера в 4 ГБ. На её основе через `Modelfile` создана **кастомная конфигурация support-bot** с собственным системным промптом. Развёртывание комплекса автоматизировано **Docker Compose**, миграции схемы — инструментом `Goose`.

Перейду к **ключевым результатам**.

По задаче анализа — сравнение пяти распространённых решений (OpenAI, Yandex GPT, Rasa, Botpress, Dialogflow CX) показало, что ни одно из них не сочетает одновременно локальное развёртывание, гибкость интеграции и отсутствие лицензионных платежей. Это обосновало разработку собственного сервиса на базе Ollama.

По задаче требований сформулированы 6 функциональных и 8 нефункциональных требований с количественными ограничениями: не более 10 сообщений за 300 секунд от одного пользователя, максимум 2000 символов на сообщение, таймаут ожидания ответа модели 60 секунд. Все ограничения закреплены декларативно в `proto`-файле и в конфигурации сервиса.

По задаче архитектуры определён минимальный интеграционный контракт всего из двух gRPC-методов — `NewChat` и `SendMessage`. Схема собственной базы данных сознательно минимизирована и состоит из двух таблиц, в которых нет сущностей системы-хоста. Благодаря этому сервис не привязан к модели данных конкретной системы и может встраиваться в произвольную систему управления заявками.

По задаче реализации микросервис построен по многослойной архитектуре с инверсией зависимостей через Uber FX. Отмечу два ключевых решения: ограничение частоты запросов реализовано SQL-фильтрацией существующих записей по временной метке — это исключает необходимость во внешнем компоненте вроде Redis; интеграция с языковой моделью построена на тонком HTTP-клиенте, что позволяет при необходимости заменить Ollama на любой OpenAI-совместимый API.

По задаче интеграции на стороне `helperbot-core` реализован прокси-репозиторий, делегирующий все вызовы AI-сервису, а на стороне Telegram-бота — конечный автомат с двумя состояниями AI-чата. Ошибка превышения лимита распознаётся клиентом средствами стандартного механизма `errors.Is` — без разбора текста сообщения.

По задаче тестирования и развёртывания разработано 10 юнит-тестов, покрывающих все ветви бизнес-логики метода `SendMessage`; все тесты успешно проходят. Развёртывание комплекса выполняется единственной командой `make compose-up`.

МИНУТА 7 — финальные выводы + последняя фраза

[Слайд 8 — Демонстрация]

Работоспособность системы подтверждается следующим **сценарием демонстрации**: запуск комплекса одной командой; переход пользователя в режим «Чат с ИИ» в Telegram-боте; отправка типового вопроса и получение ответа от модели; повторный вопрос — демонстрация контекстной осведомлённости модели.

Таким образом, **цель выпускной квалификационной работы достигнута, все поставленные задачи решены**. Разработанный сервис апробирован в составе комплекса Helperbot для ЦБС Петроградского района. Его практическая ценность — в готовности к встраиванию в произвольную систему управления заявками без передачи пользовательских данных за пределы инфраструктуры организации, что делает сервис применимым в средах с жёсткими требованиями к информационной безопасности.

Доклад окончен. Благодарю за внимание, готов ответить на ваши вопросы.